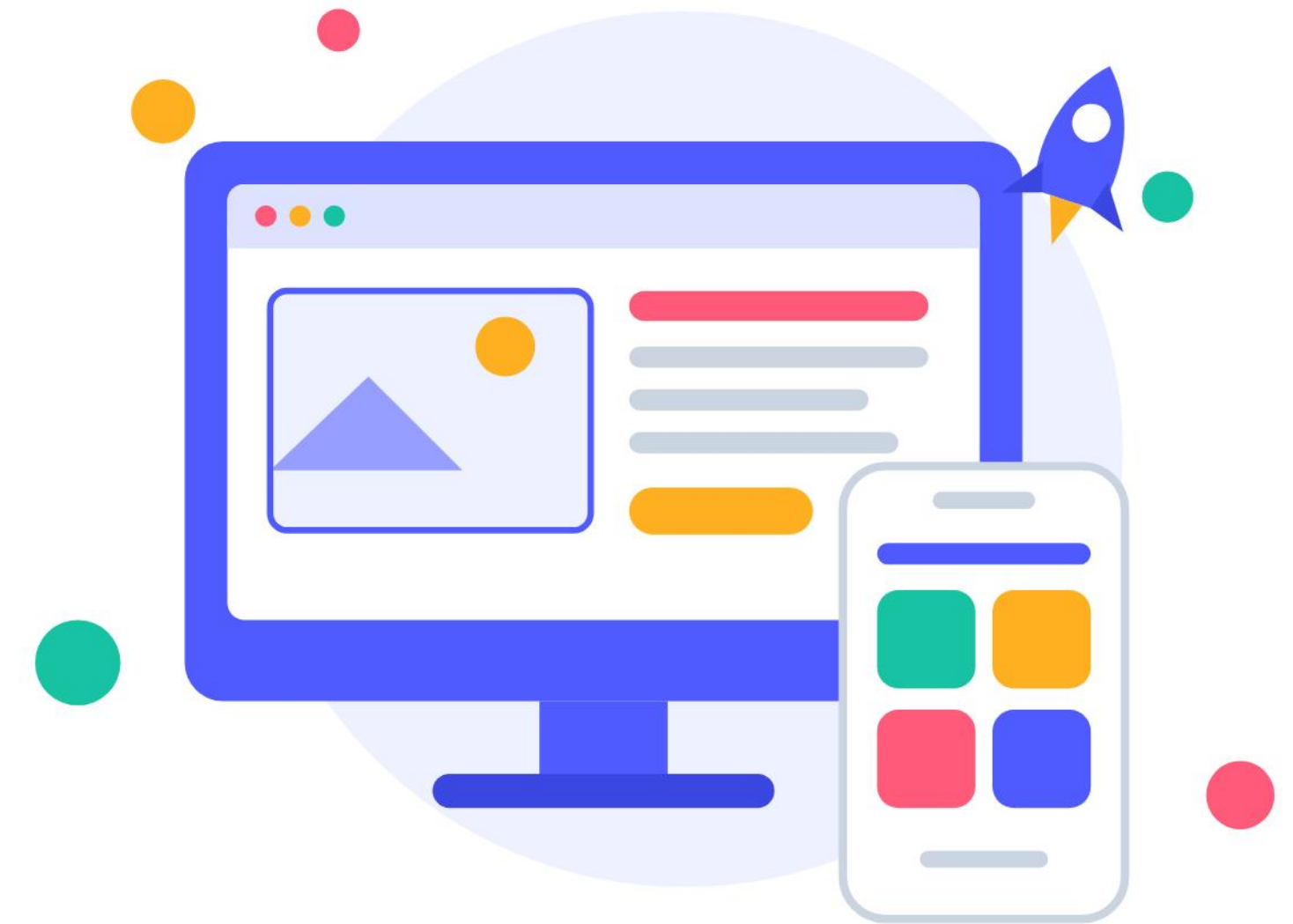


WEEK 02 · HTML II · FORMS, TABLES, AND THE READER WHO CANNOT SEE

THE READER WHO CANNOT SEE

Last week: a page a browser can parse. Today: one a person can use without a mouse or eyes.

YOSSI ARVIV · FRONT-END WEB DEVELOPMENT · RUPPIN



Access by everyone regardless of disability is an essential aspect.

TIM BERNERS-LEE

WHERE WE ARE

Week 2 of 12 (week 13 is the exam). Your week-1 page is in your repo; today it grows a form, a table and a way in.

THE NEXT 180 MINUTES

Five cycles, you typing in every one: a form · a table · the reader · one review from Claude · details and picture. Break at 90, assignment at 145.

WHAT THIS UNLOCKS

Next week the CSS goes on exactly this markup. Every label and landmark you type today is a selector then.

MINUTE 2 · THE PROBLEM

YOUR FORM LOOKS FINISHED. IT IS EMPTY.

Grey words above each box, a hint inside it, a Send button. It validates with zero errors. Three things are wrong with it, and none of them is visible.

1. Click the words. **Nothing happens** — they are not attached to anything.
2. Press Send. The address bar shows a bare `?` — **no field has a name**, so nothing was sent.
3. Close your eyes. The hint vanished the moment you typed, and the reader hears a box with a name it borrowed from the placeholder.

THIS WEEK'S RULE

Green is a floor, not the standard. The validator passed this form. A checker built into Chrome will pass it too (cycle 3, and again in cycle 4). The standard is what a person can do: click, type, hear, Tab.

Focus is the one element that gets the keyboard right now; Tab moves it to the next, in page order, and a ring shows where it is. Accessibility is graded from this week on: never delete the focus ring.

ACCESSIBILITY IS GRADED FROM THIS WEEK

- Keyboard** Tab through the whole page. Everything interactive must be reachable, in document order.
- Focus** You must be able to *see* where you are. Never delete the focus ring.
- Landmarks** `header` `nav` `main` `footer` — how a screen-reader user skips to the content; the skip link is the shortcut to `main`.
- Names** Every image, every field, every link says what it is — in words that are on the screen.

Every assignment from this one onwards has a row for this: axe with the WCAG A/AA tags, zero serious or critical. Green is the floor; the rows above it are the standard.

TARGET · MINUTE 0

THIS IS WHERE YOU WILL BE AT MINUTE 180

the class assignment – week 1's page, grown; still no CSS, on purpose

[Skip to content](#)

Maya Cohen

Third-year student at Ruppin. I already write C# and Python; this semester I am finding out what the browser does with the text I send it.

Maya Cohen

WIDE 640x120 – from 700px

- [Home](#)
- [About me](#)
- [What I am learning](#)
- [Contact me](#)
- [Longer version](#)

About me

[Skip to content](#)

Maya Cohen

Third-year student at Ruppin. I already write C# and Python; this semester I am finding out what the browser does with the text I send it.

Maya Cohen

WIDE 640x120 – from 700px

- [Home](#)
- [About me](#)
- [What I am learning](#)
- [Contact me](#)
- [Longer version](#)

About me

targets/week-02.png · 1280

[Skip to content](#)

Maya Cohen

Third-year student at Ruppin. I already write C# and Python; this semester I am finding out what the browser does with the text I send it.

375 · the narrow banner, nothing overflows

► **THE POINT** — same page as last week, plus a form every word of which is clickable, a table a screen reader can walk, a date a machine can read, a skip link, a nav that says where you are, and a banner that picks its own source. You type every piece of it today: once in a cycle, once in the assignment. The long blocks and the console one-liners are marked **COPY THIS**.

AI THIS WEEK

TUTOR. IT EXPLAINS; YOU TYPE.

LAYER · WEEKS 1–5 · PRIVATE TUTOR

ALLOWED

Ask it why a label must be tied to its field, what a table header does, what a row in a checker's report means. Ask it to review a form *you* typed. Then check the answer in the browser — click the words, press Tab — not in your gut.

NOT ALLOWED

Pasting generated markup into the assignment. **You type every piece yourself**; the long blocks and the console one-liners are marked copy this. What to check this week: a green tool score is a floor. Check the answer the way a user would — click, press Tab, look at what the tools report.

NO SUBSCRIPTION?

Everything works in the free chat, and nothing in the course depends on a paid plan. Cycle 4 is one review, twice, in whatever chat you have.

CYCLE 1 · BEFORE YOU TYPE

CLICK THE WORDS "FULL NAME"

demos/05-forms/01-broken – nothing happens

↻ reload

open ↗

Contact us

Full name

Full name

Email

Email

Message

Message

send

Press "send" to see what the server would receive.

► **HANDS-ON** demos/05-forms/ — Click the words “Full name”. Nothing. Press `send`. Nothing. Then run the validator on it — it passes. And in cycle 3 you run a checker built into Chrome on it: it gives it 100.

CYCLE 1 · BEFORE YOU TYPE

A FORM COLLECTS FIELDS. THE BUTTON SENDS THEM.

```
1 <form>
2   <input type="text" name="q" />
3   <button type="submit">Send</button>
4 </form>
```

page.html?q=alien

form

The box around fields that travel together: one Send, one request.

button

`type="submit"` sends the form. Always write the `type`.

name

The key a field travels under (`q`). No `name`, not sent.

fieldset · legend

`fieldset` groups related fields; `legend` names the group.

CYCLE 1 · BEFORE YOU TYPE

GET READS. POST CHANGES.

METHOD="GET"

The fields go **into the address**: `?q=alien&room=204`. After the `?`, `name=value` pairs joined by `&`: the **query string**. For requests that *read*: search, filter, page 2. The result can be bookmarked and shared.

METHOD="POST"

The fields go **in the body of the request**, not in the address. For requests that *change* something: sign up, send a message, pay. It needs a server to receive it: week 11.

This week every form uses `get`. The address bar is the debugger: press Send, then read what travelled.

GET OR POST

METHOD="GET"

The fields travel **in the URL**: `?q=alien&room=204`. For requests that *read* — search, filter, page 2. The result can be bookmarked, shared, refreshed, and it sits in the history. This week's debugger: Send, then read the address bar.

METHOD="POST"

The fields travel **in the request body**, not in the URL. For requests that *change* something — sign up, send a message, pay. Not bookmarkable, and the browser asks before re-sending it on refresh. Needs a server to receive it: week 11.

Neither is "more secure": both are readable in transit without TLS. The difference is meaning — read versus change — and where the data shows up.

CYCLE 1 · BEFORE YOU TYPE

THE SECOND TREE: WHAT A SCREEN READER WALKS

A **screen reader** is a program that reads a page aloud. It does not read your tags: it walks the **accessibility tree**, which the browser builds beside them. Every node has:

role	What it is: <code>textbox</code> , <code>button</code> , <code>link</code> , <code>columnheader</code> .
name	What is announced for it: <code>"Full name"</code> .
description	Extra text, read after the name.
aria-label	A name set from a string nobody sees: <code><nav aria-label="Main"></code> . Fine on a <code>nav</code> ; never on a field (cycle 4).

LOOK AT IT YOURSELF

DevTools · Elements · select a field · the **Accessibility** pane. On today's first form it says `textbox "Full name"`, and the name comes from the *placeholder*, not from the grey words you see. Making the two the same thing is cycle 1.

THE ACCESSIBILITY TREE, AND WHERE A NAME COMES FROM

Beside the DOM, a second tree for assistive technology: each node has a **role**, a **name** (what is announced), sometimes a **description**. A field's name, in order:

- 1 · **labelledby** `aria-labelledby`: the text of the elements it points at
- 2 · **aria-label** the attribute's string — invisible on screen
- 3 · **label** `label` for or a wrapping `label` — the visible words
- 4 · **title** then `placeholder`: last resorts — so axe passes a placeholder-only field

WHY THE ORDER MATTERS

Higher wins. An `aria-label` on a field with a visible label *replaces* the words in the tree: the user sees one thing, hears another. The description (`aria-describedby`) is read after the name.

Source: Accessible Name and Description Computation; HTML-AAM.

WHERE THE TREE LIVES IN DEVTOOLS

ELEMENTS · ACCESSIBILITY PANE

Select any element. The side pane (next to Styles, Computed) shows its **Name**, **Role**, **Description**, the ARIA attributes on it, and the computed properties — and the path of nodes above it.

THE FULL TREE

The accessibility icon at the top-right of the DOM tree switches the whole Elements panel to the accessibility tree: roles and names instead of tags. `columnheader`, `group`, `textbox "Email"` — this is what the reader gets.

LIGHTHOUSE

The Lighthouse tab · tick
Accessibility only · Analyze page load. Each red row names the rule and the element; "Learn more" is the axe rule's page. Free, no extension. The axe DevTools extension shows the same rules with impact levels.

In the Console (cycle 2 explains it): `document.activeElement` tells you where focus is; `$$('nav a[aria-current="page"]').length` counts what the reader will hear as current.

FOR POINTS AT ID. NAME IS WHAT GETS SENT.

for On the `label`; holds the field's `id` exactly.
Why the click works.

id Unique in the page. A pasted duplicate misdirects the click.

name The key the server receives. No `name`, no data.

type="email" `text` is a plain box. `email` `tel` `number` `date`: a fitting phone keyboard; `email`, `number` also check. `required` stops an empty Send.

radio One choice of several, one shared `name`.
`value` is what is sent; `checked` starts it selected.

WHAT THE ADDRESS BAR SHOWS AFTER SEND (METHOD="GET")

```
<input type="email" name="email" id="a" required>
<input type="text" name="name" id="b">
<input type="date" id="c">
```

?

email=maya%40ruppin.ac.il&name=May

a

The third field has no `name`. It is on the page, the user filled it in, and it is not in that string.

FORMS — THE INPUT TYPES WORTH KNOWING

text	anything short
email	validates the shape, changes the phone keyboard
tel	numeric keypad; no validation
number	with <code>min</code> , <code>max</code> , <code>step</code>
date	a real date picker (also <code>time</code> , <code>month</code>)
password · url	masked, not secure; a URL shape check
checkbox · radio	many yes/no · one of several, same <code>name</code> (also <code>file</code> , <code>range</code> , <code>color</code>)
textarea · select	elements, not types: multi-line; a list of options

The image shows a 'Create Account' form with the following fields and controls:

- Full name**: A text input field.
- Email**: An email input field.
- Newsletter topics**: Two checkboxes labeled 'Tech' and 'Deals'.
- Plan**: Two radio buttons labeled 'Basic' (selected) and 'Pro'.
- Country**: A dropdown menu with 'Choose...' as the placeholder.
- About you**: A large text area for a bio.
- Buttons**: 'Sign up' and 'Preview' buttons at the bottom.

Every field here has a real label. That is the assignment.

ID, NAME, FOR — THREE ATTRIBUTES, THREE JOBS

```

1 <label for="email">Email</label>
2 <input type="email" id="email" name="email" />
3
4 <input type="radio" id="s" name="member"
5     value="student" checked />
6 <input type="radio" id="t" name="member"
7     value="staff" />
8
9 <select id="topic" name="topic">
10   <option value="course">This course</option>
11 </select>

```

?

email=maya%40example.com&member=student&topic=course

id

For the page: unique; what `for`, `href="#..."` and `aria-describedby` point at.

name

For the server: the key in the query string. Radios share it; a `select` sends the chosen `option`'s `value`.

for

On the `label` only; must equal an `id`, character for character. Or wrap: `<label>Email <input></label>`.

value

What is sent for a radio, checkbox or option; the initial text of a text field.

REQUIRED, AND THE BROWSER'S OWN VALIDATION

```
<input type="email" required />
<input type="number" min="1" max="4" step="1" />
<input type="text" minlength="2" maxlength="40" />
<input type="text" pattern="[0-9]{9}" title="nine digits" />
<form novalidate>...</form>
```

required

An empty Send stops on the field — no JavaScript.

type

`email` refuses text without `@`;

`number` refuses letters.

limits

`min`, `max`, `step`, `minlength`, `maxlength`, `pattern` (a regex).

novalidate

On the `form`: switches it all off. Never as a way out.

The browser checks for the user; the server re-checks for you (week 11).

PLACEHOLDER IS NOT A LABEL. A HINT IS NOT A NAME.

PLACEHOLDER ONLY

```
1 <input type="text" placeholder="Email" />
```

Disappears the moment you type. Cannot be clicked. Becomes the field's name only because nothing else did.

LABEL + FOR, HINT + ARIA-DESCRIBEDBY

```
1 <label for="email">Email</label>
2 <input type="email" id="email" name="email"
3     aria-describedby="email-hint" />
4 <p id="email-hint">Only for reminders.</p>
```

`for` points at `id`: clicking the label focuses the field. The hint is the field's *description*, wired with `aria-describedby`.

placeholder is not a label, always provide a real `<label>`.

THE FORM, DONE PROPERLY

```
1 <form method="get">
2   <fieldset>
3     <legend>Your details</legend>
4
5     <label for="name">Full name</label>
6     <input type="text" id="name" name="name" required />
7
8     <label for="email">Email</label>
9     <input type="email" id="email" name="email" required />
10
11    <button type="submit">Send</button>
12  </fieldset>
13 </form>
```

No `action` means "send to this page" — with `get`, the fields land in the address bar, which is why the cycles use it as a zero-JavaScript debugger. `action="/contact"` and a server arrive in week 11. `button` always carries `type`: the validator (`no-implicit-button-type`) insists.

CYCLE 1 · 29 MIN

THE FORM THAT FAILS WELL

The film club's sign-up form: grey words, placeholders, no names. Click, Send, validate — three tools, three silences. Then one attribute at a time until every word is a handle and every field has a name.

1 I DO14 MIN 7 on the slides above, 7 in the editor · live-code/01-form-that-fails-well · label for + id + name · type + required · fieldset + legend · aria-describedby · a radio group	2 WE DO5 MIN you type it on your copy · compare with checkpoint 1 · every label focuses its field	3 YOU DO8 MIN warmup/ — the booking form from target.png · two input types you have to guess	4 DEBRIEF2 MIN placeholder is not a label · id vs name · ↗ the depth slides
--	---	--	---

YOU DO · AI-FREE

CHECKPOINT 1

YOUR SCREEN SHOULD LOOK LIKE THIS

Ruppin Film Club

One film a week, in the small hall, for anyone on campus.

- [Screenings](#)
- [Join the club](#)
- [About the club](#)

Screenings this month

Alien on Tuesday 2 November at 19:30 in room 204. The Apartment on Tuesday 9 November at 19:30 in room 204. Spirited Away on Tuesday 16 November at 19:30 in the auditorium.

Join the club

Membership is free. Fill in the form and we add you to the list for the next screening.

Your details

Full name

Email

Only for a reminder before each screening. No newsletter.

Phone

Membership

☒ Student

☐ Staff

Message

Send

Compare — `live-code/01-form-that-fails-well/checkpoint.png` is this picture at 1280; `live-code/01-form-that-fails-well/end/` is the code behind it. Every label focuses its field, an empty Send stops on Full name, and a full Send writes five keys in the address bar.

A TABLE IS A GRID OF RELATIONSHIPS

Rows (`tr`) of cells: `td` data, `th` header; `tbody` holds the rows, `thead` the header row.

BOLD CELLS

```
<thead>
  <tr>
    <td><strong>Film</strong></td>
    <td><strong>Room</strong></td>
  </tr>
</thead>
```

HEADERS WITH SCOPE

```
1 <caption>Screenings this month</caption>
2 <thead>
3   <tr>
4     <th scope="col">Film</th>
5     <th scope="col">Room</th>
6   </tr>
7 </thead>
8 <tbody>
9   <tr><th scope="row">Alien</th><td>204</td></tr>
10 </tbody>
```

Validates. The tree says cell "Film"; a reader on 204 hears "204".

`th scope="col"` names its column,
`th scope="row"` its row (rowheader Alien).
`caption` names the table.

TABLES — THE FULL ANATOMY

```
1 <table>
2   <caption>Hours</caption>
3   <thead>
4     <tr>
5       <th scope="col">Day</th>
6       <th scope="col">Hours</th>
7     </tr>
8   </thead>
9   <tbody>
10    <tr>
11      <th scope="row">Sunday</th>
12      <td>08:00–16:00</td>
13    </tr>
14  </tbody>
15 </table>
```

`scope` says which header owns which cell: `col`, `row`, `colgroup`, `rowgroup`. Also `tfoot`; `colspan` / `rowspan`; `headers="id id"` for tangled tables. Rows go inside `tbody`.

TIME: THE TEXT IS FOR PEOPLE, DATETIME IS FOR MACHINES

ONE DATE, SPELLED TWICE

2027-11-02	a date
2027-11	a month
2019	a year
19:30	a time
2027-11-02T19:30	a date and a time

```
1 <td><time datetime="2027-11-02T19:30">Tuesday 2 November,
```

A wrong format — "2 Nov 2027" — is not an error anywhere. Not in the browser, not in the validator. The machine simply gets a string it cannot read.

The text: anything a person reads. The `datetime`: one format, `YYYY-MM-DD`, then `T` and the time. Google, a calendar, and week 11's JavaScript read that one — none of them reads "Tuesday".

DATETIME — EVERY VALID FORMAT

DATETIME	WHAT IT IS
2027-11-02	a date — year-month-day, two digits each
2027-11 · 2027 · 11-02	a month · a year · a yearless date (a birthday)
19:30 · 19:30:00	a time
2027-11-02T19:30	a local date and time (a space instead of T is also valid)
2027-11-02T17:30Z · ...T19:30+02:00	a global date and time, with the offset from UTC
2027-W44 · PT2H30M	a week · a duration

Not a format: 2 Nov 2027 , 02/11/2027 , Tuesday . Nothing complains and no machine can read it — that text goes inside the element. Source: the HTML Living Standard, "the time element".

CYCLE 2 · BEFORE YOU TYPE

THE CONSOLE: TYPE ONE LINE, READ THE ANSWER

Console

A DevTools tab. You type one line of JavaScript, press Enter, and the page answers. It changes nothing in your file.

`$$('selector')` "List every element that matches this CSS selector." `.length` after it counts them.

COPY THIS – JAVASCRIPT, WEEK 8

```
$$('th[scope]').length  
// 6
```

You will not write JavaScript until week 8.
Today the Console is a question you ask the page: how many, and which one has focus.

CYCLE 2 · 25 MIN

THE TABLE THAT DESCRIBES ITSELF

Three films, three dates, three rooms, in one paragraph. It becomes a table — first the way everybody types it, with bold cells for headers, then the way a screen reader can walk it. Same pixels, different tree. Then the dates get a second spelling, for machines.

1 I DO 10 MIN 3 on the slides above, 7 in the editor · live-code/02-table-describes-itself · caption · thead + th scope="col" · tbody + th scope="row" · time datetime	2 WE DO 5 MIN you type it on your copy · compare with checkpoint 2 · columnheader and rowheader in the tree	3 YOU DO 8 MIN your own week-1 page: a real table with a caption and scoped headers, and a time on a date you already have · AI as tutor, checked in the tree	4 DEBRIEF 2 MIN td strong is a cell · who reads datetime · ↗ the depth slides
---	--	--	--

YOU DO · AI AS TUTOR

CHECKPOINT 2

YOUR SCREEN SHOULD LOOK LIKE THIS

Ruppin Film Club

One film a week, in the small hall, for anyone on campus.

- [Screenings](#)
- [Join the club](#)
- [About the club](#)

Screenings this month

Film	Date	Room
Alien	Tuesday 2 November, 19:30	204
The Apartment	Tuesday 9 November, 19:30	204
Spirited Away	Tuesday 16 November, 19:30	Auditorium

Join the club

Membership is free. Fill in the form and we add you to the list for the next screening.

Your details

Full name

Email

Only for a reminder before each screening. No newsletter.

Phone

Membership

☒ Student

☐ Staff

Compare — `live-code/02-table-describes-itself/checkpoint.png` is this picture at 1280; `live-code/02-table-describes-itself/end/` is the code behind it. In the Accessibility tree: `table "Screenings this month"` with three `columnheader` and three `rowheader` ; in the Console (copy this, JavaScript, week 8) `$$('th[scope]').length` is 6.

CYCLE 3 · BEFORE YOU TYPE

LIGHTHOUSE: CHROME'S OWN ACCESSIBILITY REPORT

- Lighthouse** A tab in DevTools. Tick **Accessibility** only, press **Analyze**. Free, no extension.
- axe** The rule engine Lighthouse runs on your page: a few dozen checks, each with a **rule id** such as `label`, `image-alt`, `heading-order`.
- tags · impact** `wcag2a` and `wcag2aa` mark rules from WCAG, the W3C's guidelines, levels A and AA. Impact: critical, serious, moderate.
- score** 0 to 100. Every failed rule is one red row that names the element. 100 means no rule failed.

WHAT A RULE CAN CHECK

That a name exists, that heading levels are in order, that an image has `alt`. Whether the name is the words on the screen, or whether clicking them works, no rule can say. Keep that in mind when the first report appears.

PRESS TAB. WHAT IS THE FIRST THING YOU REACH?

THE WAY IN

```
1 <body>
2   <a href="#main">Skip to content</a>
3   <header>...</header>
4   <nav>...
5     <a href="index.html" aria-current="page">Home</a>
6   ...</nav>
7   <main id="main" tabindex="-1">...</main>
8 </body>
```

Skip link

`href="#main"`: the first thing Tab reaches; jumps past header and nav.

`tabindex="-1"`

Lets `main` take focus (`document.activeElement` names who has it). Never `tabindex="1"`: positive means "before everything".

`aria-current`

`"page"` on the one nav link to this page: heard as "current page".

Headings

An outline, not a font size. `h2` then `h4` is a red row.

THE SKIP LINK, LINE BY LINE

```
1 <body>
2   <a href="#main">Skip to content</a>
3   <header>...</header>
4   <nav>...</nav>
5   <main id="main" tabindex="-1">...</main>
```

first

First in `body`, so the first Tab reaches it.

href

A fragment link: the target is an `id` on the same page.

tabindex

Measured: with `-1`, `activeElement` is `main` and the ring shows; without it focus stays on `body` — Chromium still continues from `main`, but assistive technology follows focus.

visible

With no CSS it is the first line of the page. From week 3: hidden until focused — never `display: none`.

ARIA-CURRENT — THE VALUES

page	the link to the page you are on — this week's value, exactly one per page
step	the current step of a wizard or a checkout
location	the current place in a breadcrumb or a map
date	today in a calendar
time	the current slot in a timetable
true · false	current, with no more specific meaning; absent

ONE FACT, ONE PLACE

The attribute is the truth: a screen reader announces "current page", and next week `nav a[aria-current="page"]` is the selector that styles it. A `class="active"` would be a second copy of the same fact to keep in sync. Source: WAI-ARIA 1.2, `aria-current`.

HEADINGS ARE AN OUTLINE

WHAT A SCREEN READER LISTS

```
<h1>Ruppin Film Club</h1>
  <h2>Screenings this month</h2>
  <h2>Join the club</h2>
  <h2>About the club</h2>
    <h3>Where we meet</h3>
```

one h1

The page's title. The assignment grades exactly one.

no skipped level

After h2 comes h3. h4 "because h3 looked too big" is the heading-order row.

not a font size

A heading is a level in the table of contents a screen reader offers (VoiceOver's rotor, NVDA's H key). Size is CSS, next week.

sections

Every section and article gets a heading; aside too if it has content of its own.

CYCLE 3 · 29 MIN

THE READER WHO CANNOT SEE

Same page, no mouse, no eyes. The first accessibility report, its red rows read aloud and fixed; a skip link and a main that can take focus; a nav that says where you are; and one "fix" for the Tab order that breaks everything it touches.

1 I DO11 MIN 3 on the slides above, 8 in the browser · live-code/03-reader-who-cannot-see · Lighthouse · h4 to h3 · skip link + main id + tabindex="-1" · tabindex="1" and why not · aria-current="page"	2 WE DO6 MIN you type it on your copy, Lighthouse once, after · compare with checkpoint 3 · first Tab is Skip to content	3 YOU DO10 MIN your own week-1 site, both pages: skip link, main, aria-current, Lighthouse, every red row read	4 DEBRIEF2 MIN what the report saw, what it cannot · ↗ the depth slides
--	--	--	---

YOU DO · AI-FREE

THE FIRST REPORT. GREEN IS A FLOOR.

DevTools · Lighthouse · Accessibility · Analyze. One row per axe rule id (measured, Lighthouse 12.8):

heading-order	"Heading elements are not in a sequentially-descending order" — the <code>h4</code> . Score 93.
tabindex	"Some elements have a [tabindex] value greater than 0" — serious.
label · image-alt	A field with no name at all; an <code>img</code> with no <code>alt</code> — critical.
target-size	WCAG 2.2, needs CSS. Stays red; the grader (<code>wcag2a</code> , <code>wcag2aa</code>) does not count it.

WHAT IT CANNOT SEE

The placeholder-only form: **100.** `aria-label` under words you cannot click: **100.** Bold `td` s as headers: nothing.

It checks that a name exists. Not that the name is the words on the screen, or that clicking them works. That is you: click, Tab, the tree.

CHECKPOINT 3

YOUR SCREEN SHOULD LOOK LIKE THIS

[Skip to content](#)

Ruppín Film Club

One film a week, in the small hall, for anyone on campus.

- [Home](#)
- [Screenings](#)
- [Join the club](#)
- [About the club](#)

Screenings this month

Film	Date	Room
Alien	Tuesday 2 November, 19:30	204
The Apartment	Tuesday 9 November, 19:30	204
Spirited Away	Tuesday 16 November, 19:30	Auditorium

Join the club

Membership is free. Fill in the form and we add you to the list for the next screening.

Your details

Full name

Email

Only for a reminder before each screening. No newsletter.

Phone

Membership

☒ Student

☐ Staff

Compare — `live-code/03-reader-who-cannot-see/checkpoint.png` is this picture at 1280; `live-code/03-reader-who-cannot-see/end/` is the code behind it. First Tab: Skip to content. Enter, then Tab: the Full name field. Lighthouse: one red row, `target-size`, and everybody knows why.

THE AXE RULES YOU SAW THIS WEEK

RULE ID	IMPACT · TAG	WHAT TRIGGERED IT
LABEL	critical · wcag2a	a field with no name at all (Lighthouse 91)
IMAGE-ALT	critical · wcag2a	an <code>img</code> without <code>alt</code>
HEADING-ORDER	moderate · best-practice	<code>h2</code> then <code>h4</code> (cycle 3 start, Lighthouse 93)
TABINDEX	serious · best-practice	<code>tabindex="1"</code> on the Send button (Lighthouse 91)
LABEL-TITLE-ONLY	serious · best-practice	a field whose only name is its <code>title</code>
LANDMARK-ONE-MAIN · REGION	moderate · best-practice	the bare demo form with no <code>main</code>
TARGET-SIZE	serious · wcag22aa	nav links with no spacing — every CSS-less page

The grader runs axe-core with `wcag2a` and `wcag2aa`, serious and critical only — so `heading-order` and `tabindex` get their own rows. Measured: axe-core 4.13, Lighthouse 12.8; the placeholder-only form scores 100.

WCAG 2.2 – THE ROWS THIS WEEK TOUCHES

CRITERION	LEVEL	WHAT YOU TYPED FOR IT
1.1.1 NON-TEXT CONTENT	A	<code>alt</code> on every <code>img</code> , including the one in a <code>picture</code>
1.3.1 INFO AND RELATIONSHIPS	A	<code>label for</code> , <code>fieldset/legend</code> , <code>th scope</code> , <code>caption</code> , headings
2.1.1 KEYBOARD · 2.4.1 BYPASS BLOCKS · 2.4.3 FOCUS ORDER	A	everything reachable by Tab, in document order; the skip link; no positive <code>tabindex</code>
2.4.6 HEADINGS AND LABELS · 2.4.7 FOCUS VISIBLE	AA	headings and labels that say what the thing is; the ring on <code>main</code>
2.5.8 TARGET SIZE (MINIMUM)	AA	24×24 CSS px or spacing — new in 2.2; <code>target-size</code> ; needs CSS
3.3.2 LABELS OR INSTRUCTIONS	A	a label on every field; the hint with <code>aria-describedby</code>
4.1.2 NAME, ROLE, VALUE	A	every control has a name; <code>aria-current</code> as state

4.1.1 Parsing was dropped in 2.2; validity is our rule, not WCAG's.

BREAK

10 MINUTES

Back at 100. Cycle 4 is the AI cycle — have a chat window open, free or paid, Live Server running, and DevTools on Lighthouse.

CYCLE 4 · 25 MIN

"REVIEW MY FORM FOR ACCESSIBILITY — IS ARIA-LABEL ENOUGH?"

A student who ran Lighthouse, read one thing about `aria-label`, and asks the chat to confirm it. What comes back is confident, correct about the tool it quotes, and answers the question exactly as asked. Then the room clicks the words.

1 I DO

7 MIN

2 on this slide, 5 in the chat · ai-lab/04-review-my-form · P1 live, then RUN the answer: Lighthouse 100, click the words, the tree · transcript.md if the room got something else

2 WE DO

6 MIN

everyone pastes P1 · run what came back · find what the tool did not check, before it is named

3 YOU DO

10 MIN

write P2 as a checkable constraint · run it · one question about your own form · the journal line

4 DEBRIEF

2 MIN

a leading question gets the answer it asked for · ↗ the journal exemplar

CYCLE 4 · THE GAP

AXE PASSES. LIGHTHOUSE SAYS 100. NOW CLICK THE WORDS.

WHAT THE TOOL CHECKS

That every field has an accessible **name**. `aria-label="Full name"` is a name; so is `aria-labelledby`, which points at other text on the page and borrows it as the name. So is a placeholder, when nothing else is there. Rule `label`: green.

WHAT IT DID NOT CHECK

That the name is the words on the screen. That clicking them focuses the field (it does not — `activeElement` stays `body`). That the hint survives typing (it does not). Three measurements, none of them in the report.

THE CONSTRAINT, NOT THE QUESTION

"Is aria-label enough?" is a question the answer can agree with. "Every field gets a visible `label` for ; clicking the words must focus the field" is a constraint you can check. Your journal line: *claim · how I checked · verdict.*

CHECKPOINT 4

THREE QUESTIONS BEFORE THE NEXT CYCLE

1. Did you **run** the answer to P1 — Live Server, Lighthouse, then a click on the words — instead of stopping at the 100?
2. Does your P2 state a constraint the browser can check — "clicking the words focuses the field", "label for matches the id" — and does what came back meet it, on all four words?
3. Does your journal line name **a tool and a measurement** — "Lighthouse 100", "click: nothing", "Name from aria-label" — and not just "Claude was right / wrong"?

Compare — `ai-lab/04-review-my-form/checkpoint-he.md` has the same three questions and the measurements; `artifact/claude-p1/` and `artifact/claude-p2/` are the two forms, ready to open side by side.

ARIA'S FIVE RULES

1. **Do not use ARIA** if a native element has the meaning and behaviour: `button`, not `<div role="button">`; `label`, not `aria-label`.
2. **Do not change native semantics** unless you really have to.
3. **Every interactive ARIA control must work with the keyboard.** A `div` with a role gets no Enter, Space or focus for free.
4. **Never hide something focusable** with `role="presentation"` or `aria-hidden="true"`.
5. **Every interactive element needs an accessible name.**

This week's three ARIA attributes only add to native elements:

`aria-label` on a `nav`, `aria-describedby` for a hint, `aria-current` for the page. Source: W3C, "Using ARIA".

THE VALIDATOR, AXE, AND A CLICK — WHO SEES WHAT

THE MISTAKE	HTML-VALIDATE 9	AXE / LIGHTHOUSE	THE BROWSER, BY HAND
DUPLICATE ID	no-dup-id, form-dup-name	silent	click Email — focus lands in Full name
FOR = THE NAME	silent	label critical	click — nothing
PLACEHOLDER ONLY · ARIA-LABEL	silent	silent (100)	click — nothing; typing hides the hint
BOLD TDS AS HEADERS	silent	silent	the tree says cell, not columnHeader
H2 THEN H4	silent	heading-order	a hole in the headings list
TABINDEX="1"	silent	tabindex serious	first Tab lands on Send
DATETIME="2 NOV 2027"	silent	silent	the machine reads nothing

Every row measured. No single tool sees the whole column — so the self-check lists things that *happen*, not things you wrote.

DETAILS OPENS ITSELF. PICTURE CHOOSES ITSELF. IMG IS LAST.

OPENS AND CLOSES, SPACE OR CLICK

```
1 <details>
2   <summary>How screenings work</summary>
3   <ul>...</ul>
4 </details>
```

`summary` is what you always see, and it is in the Tab order by itself. Open it and the browser writes `open` on the `details`; write it yourself to start open.

SOURCE FIRST, IMG LAST

```
1 <picture>
2   <source media="(min-width: 700px)"
3     srcset="images/banner-wide.svg"
4     width="640" height="160" />
5   
7 </picture>
```

`media` asks about the window width; `srcset` names the file to use when it matches. Each `source` is tried in order, stopping at the `img` (validator: `element-permitted-order`). The `img` holds `alt`; `width` / `height` reserve the space.

DETAILS AND SUMMARY — THE ATTRIBUTES

```
1 <details open>
2   <summary>How screenings work</summary>
3   <p>Doors open at 19:15.</p>
4 </details>
5 <details name="faq">...</details>
```

open

Present: expanded. The browser toggles it itself; write it to start open.

summary

First child. Omit it and the browser shows "Details". Focusable; Space or Enter toggles.

name

Same `name` on several `details`: opening one closes the others (every browser since 2024).

tree

Chromium: `details` is a `group` with no name of its own; the name sits on the `summary` (a disclosure triangle). The hidden content is not in the tree until it opens.

PICTURE VS SRCSET AND SIZES

ART DIRECTION – A DIFFERENT PICTURE PER WIDTH

```
1 <picture>
2   <source media="(min-width: 700px)"
3     srcset="wide.svg" width="640" height="160" />
4   <source type="image/avif" srcset="narrow.avif" />
5   
7 </picture>
```

`source` s first, in order; the first whose `media` matches and whose `type` the browser knows wins. `img` last, always: it is the fallback, the element in the tree, and the owner of `alt`.

RESOLUTION SWITCHING – THE SAME PICTURE, MORE PIXELS

```
1 
```

No `picture` needed. `srcset` lists candidates with their real widths; `sizes` tells the browser how wide the image will be laid out, and it picks the smallest candidate that is sharp enough for the screen's pixel density. Week 5.

CYCLE 5 · 20 MIN

TWO THINGS THAT MOVE, ZERO LINES OF JAVASCRIPT

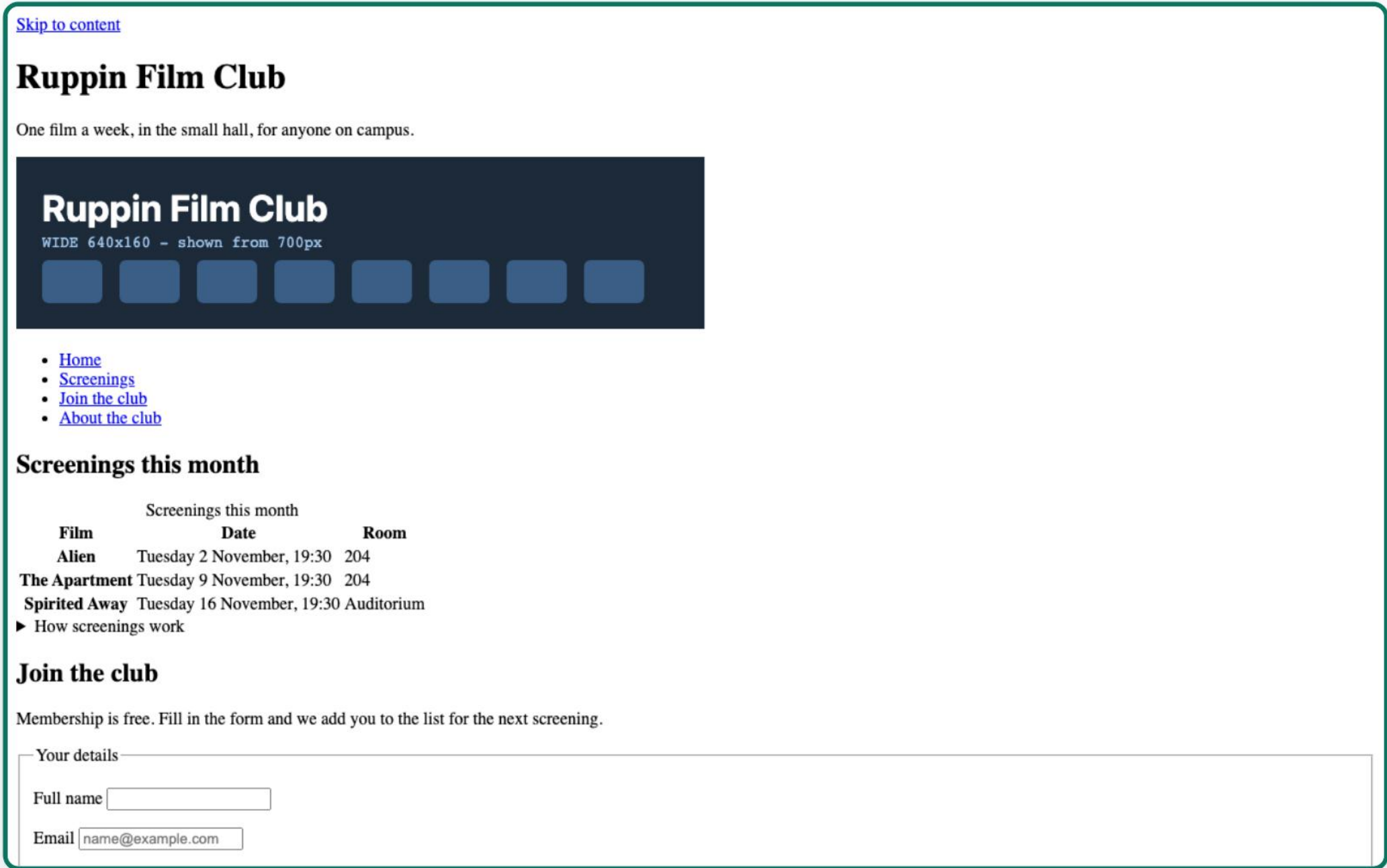
A section that opens and closes, and a banner that picks its own source when the window shrinks. Both are HTML. Both work with the keyboard, in a screen reader, and on the day your JavaScript throws.

1 I DO7 MIN 2 on the slides above (details and summary · picture and source), 5 in the editor · live-code/05-behaviour-without-javascript · details + summary · picture: source first, img last	2 WE DO5 MIN you type it on your copy · compare with checkpoint 5 · WIDE at 1280, NARROW at 375, no scrollbar	3 YOU DO6 MIN your own page: one details, one picture with two sources · AI as tutor, checked at two widths and in the validator	4 DEBRIEF2 MIN which element the reader meets · what happens when the JS breaks · ↗ the depth slides
---	---	--	--

YOU DO · AI AS TUTOR

CHECKPOINT 5

YOUR SCREEN SHOULD LOOK LIKE THIS



Compare — `live-code/05-behaviour-without-javascript/checkpoint.png` is this picture at 1280; `live-code/05-behaviour-without-javascript/end/` is the code behind it. Device toolbar at 375: the NARROW banner and no scrollbar. Space on the summary: the list opens.

ASSIGNMENT · 145–175

THE ASSIGNMENT: ASSEMBLE WHAT YOU TYPED, THEN EXTEND IT

ASSEMBLE · CORE

Your week-1 page, copied into `week-02/`: the contact form with `fieldset`, `label` for, `name`, two input types and `required` (cycle 1) · a table with `caption` and `th scope`, a `time datetime` (cycle 2) · skip link, `main id tabindex="-1"`, `aria-current="page"`, headings in order, Lighthouse read, validator clean, no `CODE HERE` left (cycle 3). At least three commits.

EXTEND · STRETCH AND CHALLENGE

A hint with `aria-describedby`, a `details`, a `picture` with the `img` last, `th scope="row"`, and `about.html` with the same three lines (cycles 1, 2, 3, 5). Then: the picture fits at 375, a radio group, and no `aria-label` on fields — the words on the screen are the label (cycle 4).

THE RULES, AND THE EXAM

No CSS. No JavaScript. No `aria-label` on fields. No positive `tabindex`. AI as tutor — it reviews, you type, you click. Submit the repo URL and the SHA in Moodle. The exam in week 13 asks you about this code, offline.

DEBRIEF · 175–180

THREE QUESTIONS BEFORE YOU LEAVE

1. A form validates clean and Lighthouse says 100. Name two things that can still be wrong with it, and the test for each.
2. A screen reader lands on the cell `204`. What does it say in a table of bold `td`s, and what does it say once the headers are `th scope`?
3. You press Tab on a fresh page. What should you reach first, what attribute lets `main` take focus, and which attribute value would break the whole order?

► **GO DEEPER** — every slide marked *Deeper · self-study* is optional depth and is not examined: every input type, get or post in full, every datetime format, how a field's accessible name is computed, ARIA's five rules, the WCAG 2.2 rows this week touches, the axe rule ids you saw in the report, and one table of who sees what. Each one sits after the slide it extends.